# Лабораторна робота № 3
# Управляючі конструкції в мові Go. Функції. Введення та виведення інформації у консоль

**Мета роботи**: ознайомитися з управляючими конструкціями мови програмування Go, навчитися користуватися функціями вводу/виводу та створювати користувацькі функції. Ознайомитись з методами генерування випадкових чисел.

## Короткі теоретичні відомості

У мові Go є чотири керуючі конструкції: *if*, *switch*, *for*, *select*.

Перші три керуючі конструкції дуже схожі з керуючими конструкціями у мовах C/C++, тому розглянемо їх коротко. Останню конструкцію розглянемо при вивченні паралельного програмування.

Умовні вирази дозволяють створювати код, який не буде/буде виконуватиметься в залежності від умов в програмі. Оператор *if* в мові Go має наступний вигляд:

*if [необов'язкова_інструкція1;] <логічний_вираз1> {*

*блок1*

*} else if [необов'язкова_інструкція2;] <логічний_вираз1> {*

*блок2*

*} else {*

*блок3*

*}*

де *необов'язкова_інструкція1*, *необов'язкова_інструкція2* – використовуються для отримання деякого результату, який буде використовуватись у середині конструкції *if*; *логічний_вираз1*, *логічний_вираз2* – умова (вираз), яка повертає значення типу *bool*. Якщо ця умова істина, то виконується наступний блок інструкцій за умовою.

Опціонально, за *if* може слідувати оператор *else if*, за яким може бути завершальний *else*, що зручно для перевірки декількох умов. Оператор *if* може не мати/мати оператор else, який повинен бути останнім в ланцюжку *else if*. При

цьому *if* може мати будь-яку кількість операторів *else if*, але всі вони повинні передувати оператору *else*.

Наприклад:

```
a := 6
    b := 7
    if a < b {
        fmt.Println("a < b")
    }
```

Даний приклад показує застосування тільки оператору *if*. Наступний приклад демонструє застосування усіх інструкцій.

```
if _, err := os.Stat("file"); err == nil {
    fmt.Println("unknown file exists")
} else if _, err := os.Stat("/home/myfile.go"); err == nil {
    fmt.Println("real file exists") // Fourth: real file exists
} else {
    fmt.Println("both files not exist")
}
```

Цикли дозволяють в залежності від певної умови виконувати деякі дії безліч разів. У Go є тільки один цикл – *for*, який може приймати різні форми. Цей цикл має наступне визначення:

*for [[інструкція;] [лічильник]; [умова]; [зміна лічильника]] {*

*блок*

*}*

де *інструкція* – необов'язкова інструкція, використовуються для отримання деякого результату, який буде використовуватись у середині конструкції *for*; умова –вираз, який повертає значення типу bool. Якщо ця умова істина, то виконується блок інструкцій *for*; *лічильник, зміна лічильника* – відповідають за кількість ітерацій циклу.

Оператор *break* та *continue* аналогічні мові C/C++.

Приклад:

```
first := 10
for { // Нескінченний цикл з перериванням циклу за допомогою break
    fmt.Print(".")
    if first <= 0 {
        break
```

```
    }
    first--
}
```

Приклад:

```
i := 1
for i <= 10 { // Аналог циклу while
    fmt.Println(i)
    i = i + 1
}
```

Приклад:

```
for second := 10; second < 20; second++ { // Класична реалізація циклу for
    fmt.Print(" .")
}
```

В мові Go цикл *for* має ще одну форму:

*for ключ, значення := range об'єкт { // Ітерації по об'єкту*

*блок*

*}*

Наприклад:

```
third := []string{"a", "b", "c", "d", "e", "f"}
for index, val := range third {
    fmt.Println(index, " -> ", val)
}
```

Існують два різновиди інструкції вибору *switch*: вибір за значенням вирази і вибір за типом.

Інструкція *switch* вибору за значенням виразу:

*switch [інструкція;] [умова] {*

*case вираз1:*

*блок1*

*...*

*case виразN:*

*блокN*

*default:*

*блокN+1*

*}*

де *інструкція* – необов'язкова інструкція, використовуються для отримання деякого результату, який буде використовуватись у середині конструкції *switch*; умова –вираз, який повертає значення. Якщо отримане значення відповідає виразу *case*, то буде виконується його блок інструкцій, а інакше буде виконано блок інструкцій секції *default*.

Наприклад:

```
switch {
case 1 < 2:
    fmt.Println("1 < 2")
case 2 > 3:
    fmt.Println("2 > 3")
}
```

Наступний приклад показує, що *switch* може використовувати майже любий тип даних. Також слід відмітити, що на відміну від C/C++ в Go виконується тільки один *case*. Якщо потрібно щоб виконувались декілька *case*, необхідно використати ключове слово "*fallthrough*".

Приклад:

```
switch suffix := GetExt(); suffix {
case "png":
    fallthrough
case "jpg":
    fallthrough
case "gif":
    fmt.Println("Image")
default:
    fmt.Println("Not image")
}
```

Інструкція *switch* вибору за типом:

*switch перемикач_типу {*

*case тип1:*

*блок1*

*...*

*case типN:*

*блокN*

*default:*

*блокN+1*

*}*

Наприклад:

```
var someObject interface{}
someObject = 5
switch someObject.(type) {
case bool:
    fmt.Println("bool")
case int:
    fmt.Println("int")
case string:
    fmt.Println("string")
default:
    fmt.Println("Sorry, I don't know")}
```

Функцією – називається іменована група операторів, які вирішують певне завдання. Функцію можна запустити, викликавши по імені, будь-яку кількість разів з різних місць програми.

Функції в мові Go можуть повертати декілька змінних. Часто остання змінна зберігає ідентифікатор помилки.

Визначення функцій:

*func ім'я (параметри) тип_параметра_що_повертається {*

*тіло*

*}*

*або*

*func ім'я (параметри) (параметри_що_повертається) {*

*тіло*

*}*

Ім'я - унікальний для даного простору імен ідентифікатор функції.

Наприклад:

```
func main() { // Головна функція
    . . .
}
// Функція має два вхідних параметри та одне значення повертає
func pow(number int, degree uint) int {
    . . .
```

```
}
// Функція має два вхідних параметри та ничого не повертає
func print(funcName, result string) {
    . . .
}
// Функція має один вхідний параметри, та два значення повертає
func getMap(name string) (int, bool) {
     . . .
}
```

Існує велика кількість бібліотечних функцій. Розглянемо функції для форматованого виведення та введення інформації у/з консоль.

Функція *fmt.Printf()* призначена для виведення інформації за заданим форматом.

Синтаксис:

*fmt.Printf(«Рядок виводу»[, аргумент1[, аргумент2, [...]]])*

Першим параметром функції є, як правило текст та формат виводу значень змінних, імена яких вказані в параметрах, що залишилися.

Наприклад:

```
fmt.Printf("Мама мила раму");
```

Формати виводу значень змінних задаються за допомогою специфікатора формату – послідовність символів, що починаються зі знаку %.

При виводі числових значень найчастіше використовуються наступні специфікатори:

*%d* – для виводу цілих чисел;

*%f* – для виводу чисел з плаваючою комою;

*%e* – для виводу чисел з плаваючою комою в е-формі;

*%s* – для виводу строкових змінних;

*%v* – формат виводу за замовчуванням;

*%T* – для виводу типу змінних;

тощо.

Наприклад:

```
fmt.Printf("a=%d b==%d c==%d ", a, b, c);
```

Кожна специфікація формату має вид (параметри в квадратних дужках необов'язкові):

*%[flags][width][.prec]type*,

де *%* – символ %;

*flags* – ознака вирівнювання:

‘+’ – завжди виводить знак результату;

‘-’ – вирівнювання по лівому краю, інакше вирівнювання по правому краю;

width (ціле число) – загальна ширина поля. Якщо це число починається з цифри 0, виведення доповнюється ліворуч нулями до заданої ширини. У задану ширину входять всі символи виведення, включаючи знак, дробову частину тощо.

type – тип специфікації:

d – ціле десяткове число,

x – ціле 16-ричне число,

f – число з плаваючою точкою, e - число в експоненціальній формі,

c – один символ,

s – рядок;

prec (ціле число) – кількість знаків після крапки при виведенні чисел з плаваючою точкою.

Для введення інформації використовується функція *fmt.Scanf()*.

Синтаксис:

*fmt.Scanf(«Рядок формату»,&аргумент1[,&аргрумент2[, ...]])*

Наприклад:

```
fmt.Scanf("%d", &a)
```

*Випадкова величина* – це величина, яка в результаті випробувань може приймати певні значення (із сукупності своїх значень) з певною ймовірністю. Випадковою можна назвати будь-яку (не обов'язково чисельну) змінну $x$, значення якої x створюють множину випадкових елементарних подій $\{x\}$.

Мова Go має бібліотеку math/rand для генерування випадкових чисел. Однак для вивчення роботи з керуючими конструкціями та функціями, створимо свою функцію генерування випадкових чисел.

Рядок розподілу дискретної випадкової величини $x$ може бути представлений як у табличній формі (у вигляді таблиці, де перераховано значення випадкової величини $x_1$, $x_2$, $x_n$ з відповідними до них ймовірностями $p_1$, $p_2$, $p_n$ (див. таблицю 3.1)), так і у вигляді графічного зображення (рисунок 3.1).

Таблиця 3.1

| Значення ( $x_i$ ) | $x_1$ | $x_2$ | $x_3$ | ... | $x_n$ |
|---|---|---|---|---|---|
| Ймовірності ( $p_i$ ) | $p_1$ | $p_2$ | $p_3$ | ... | $p_n$ |

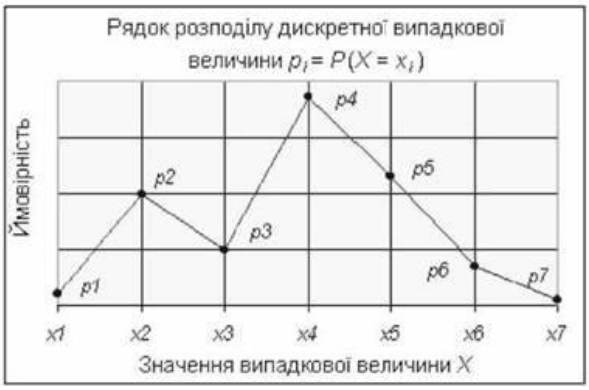


*Рисунок 3.1*

Частота інтервалів – число, що показує скільки разів значення, що потрапляють в кожен інтервал групування, зустрічається у вибірці.

На практиці часто доводиться мати справу з статистичною ймовірністю. Її часто називають відносною частотою появи події і позначають

$$P = L/K,$$

де $L$ - кількість випробувань, в яких з'явилась певна подія,

$K$ - загальна кількість випробувань.

*Математичне сподівання (середнє значення)* — одна з основних числових характеристик кожної випадкової величини. Воно є узагальненим поняттям середнього значення сукупності чисел на той випадок, коли елементи множини значень цієї сукупності мають різну "вагу", ціну, важливість, пріоритет, що є характерним для значень випадкової змінної:

$$M(X) = \sum_{i=1}^{n} x_i p_i$$

Дисперсією випадкової величини X називають математичне сподівання квадрата відхилення випадкової величини від її математичного сподівання:

$$D(X) = \sum_{i=1}^{n} \left(x_i - M(X)\right)^2 p_i$$

Числову характеристику закону розподілу випадкової величини

$$\sigma(X) = \sqrt{D(X)},$$

називають середньоквадратичним відхиленням або стандартним відхиленням.

*Лінійний конгруентний метод*. Для отримання послідовності випадкових значень використовують наступний рекурентний вираз:

$$X_{k+1} = (aX_k + c) \bmod m$$

де $m$ – модуль, $m > 0$, $a$ – множник, $0 \le a < m$, $c$ – приріст, $0 \le c < m$, $X_0$ – початкове значення, $0 \le X_0 < m$.

Модуль вибирають достатньо великим, оскільки період не може містить менше $m$ чисел. В якості $m$ рекомендується брати найбільше просте число, яке не перевищує розрядність машинного слова.

Вибір множника визначається наступною теоремою:

лінійна конгруентна послідовність, визначена числами $m$, $a$, $c$ і $X_0$ має період $m$ тоді і лише тоді, коли виконуються наступні три умови:

1) числа $c$ і $m$ є взаємно простими;

2) число $b = a - 1$ є кратним числу $p$ для кожного простого числа $p$, яке є дільником числа $m$;

3) число $b$ є кратним 4, якщо число $m$ є кратним 4.

## Порядок виконання роботи

1. Розробити функцію генерування цілочислової послідовності псевдовипадкових значень (за допомогою конгруентного методу) та виконати обробку отриманого масиву даних наступним чином (таблиця 3.2):

– розрахувати частоту інтервалів появи випадкових величин (інтервал дорівнює 1);

– розрахувати статистичну імовірність появи випадкових величин;

– розрахувати математичне сподівання випадкових величин;

– розрахувати дисперсію випадкових величин;

– розрахувати середньоквадратичне відхилення випадкових величин.

2. Розробити функцію генерування послідовності псевдовипадкових дійсних значень (за допомогою конгруентного методу) (таблиця 3.2).

Таблиця 3.2 – Вхідні данні

| Варіант | Коефіцієнти | | | Діапазон випадкових величин, n | Довжина послідовності чисел, K |
|---|---|---|---|---|---|
| | a | c | m | | |
| 1 | 1103515245 | 12345 | $2^{31}$ | [0, 100) | 10000 |
| 2 | 1664525 | 1013904223 | $2^{32}$ | [0, 150 | 20000 |
| 3 | 16807 | 0 | $2^{31}$-1 | [0, 200) | 30000 |
| 4 | 22695477 | 1 | $2^{32}$ | [0, 250) | 40000 |
| 5 | 48271 | 0 | $2^{31}$-1 | [0, 300) | 50000 |
| 6 | 214013 | 2531011 | $2^{32}$ | [0, 100) | 50000 |
| 7 | 2147483629 | 2147483587 | $2^{31}$-1 | [0, 150) | 40000 |
| 8 | 65539 | 0 | $2^{32}$ | [0, 200) | 30000 |
| 9 | 1140671485 | 12820163 | $2^{24}$ | [0, 250) | 20000 |
| 10 | 69069 | 1 | $2^{32}$ | [0, 300) | 10000 |
| 11 | 1103515245 | 12345 | $2^{31}$ | [0, 300) | 50000 |
| 12 | 1664525 | 1013904223 | $2^{32}$ | [0, 250) | 40000 |
| 13 | 16807 | 0 | $2^{31}$-1 | [0, 200) | 30000 |
| 14 | 22695477 | 1 | $2^{32}$ | [0, 150) | 20000 |
| 15 | 48271 | 0 | $2^{31}$-1 | [0, 100) | 10000 |
| 16 | 214013 | 2531011 | $2^{32}$ | [0, 300) | 10000 |
| 17 | 2147483629 | 2147483587 | $2^{31}$-1 | [0, 250) | 20000 |
| 18 | 65539 | 0 | $2^{32}$ | [0, 200) | 30000 |
| 19 | 1140671485 | 12820163 | $2^{24}$ | [0, 150) | 40000 |
| 20 | 134775813 | 1 | $2^{32}$ | [0, 100) | 50000 |

## Контрольні питання

1. Які керуючі конструкції існують в мові Go?
2. Як організувати цикли?
3. Призначення керуючої конструкції *for*?
4. Призначення керуючої конструкції *if*?
5. Призначення керуючої конструкції *switch*
6. Як здійснюється форматований вивід даних?
7. Який має синтаксис функція *fmt.Printf()*?
8. Який має синтаксис функції *fmt.Scanf()*?
9. В якому пакеті знаходяться функції введення та виведення?
10. За допомогою якої функції виводиться результат у консоль?

За допомогою якої функції можна виконати введення з клавіатури?